

LENS: LLM-guided Environment Simplification for Planning and Control in Clutter

Aileen Liao

Rachel Holladay

Dinesh Jayaraman

Michael Posa

University of Pennsylvania

{aliao, rhollada, dineshj, posa}@seas.upenn.edu

Abstract: Despite recent advances in general-purpose robotic manipulation, real-world multi-object clutter remains challenging to handle for today’s prevalent approaches. The problem scales in complexity due to more objects and collisions, more unpredictable contact physics, distractors, and task ambiguity. Bridging this gap to real-world deployment requires effective scene abstractions; yet today, producing such abstractions requires extensive task-specific manual engineering, which does not scale. These abstractions are costly to generate and difficult to adjust or fine-tune. We instead propose a plug-and-play fix to automatically generate scene-specific, task-specific, adaptively updating abstractions on top of existing planning and control stacks. **LLM-guided Environment Simplification (LENS)** produces a de-cluttered abstracted scene representation by merging (e.g., stacked objects) or pruning (e.g., distant objects) scene entities in a closed loop in response to task progress. These dynamic, task-relevant abstractions are versatile and easy to use. In our experiments, we show that LENS improves classical planning, model-based control, and a vision-language-action model, across a diverse set of highly cluttered manipulation scenes. Project website: <https://lens-2026.github.io/>.

Keywords: Robot Learning, Manipulation, Planning and Control

1 Introduction

A messy home is a happy home, but a robot’s nightmare. As scene clutter grows in unstructured real-world environments, the number of objects, contacts, decision branches, and interaction modes become limiting factors for solvability and real-time performance in planning and model-based control. Learning offers no silver bullet here; learned policies face difficulty in cluttered scenes due to semantic and spatial ambiguity, as well as distribution shifts. So, current research often focuses on curated environments with few, task-relevant objects, bearing little resemblance to the real-world.

This challenge cuts across the full robot control spectrum. Optimization- and planning-based methods must reason over every object in the scene, so their computational cost and failure modes scale with clutter even when most objects are task-irrelevant. Learned controllers, including modern vision-language-action models, face a parallel difficulty: trained on tidy, curated scenes, they encounter severe distribution shift when deployed in dense, unstructured environments, leading to degraded affordance estimates and incorrect contact predictions. In both regimes, performance collapses as scenes grow complex and with no way to focus on what matters. Without a flexible mechanism to suppress task-irrelevant objects before planning or control begins, the combinatorial and distributional burdens imposed by clutter are inherited in full by every downstream method, making the gap between lab and real-world not merely a matter of scale but of fundamental tractability.

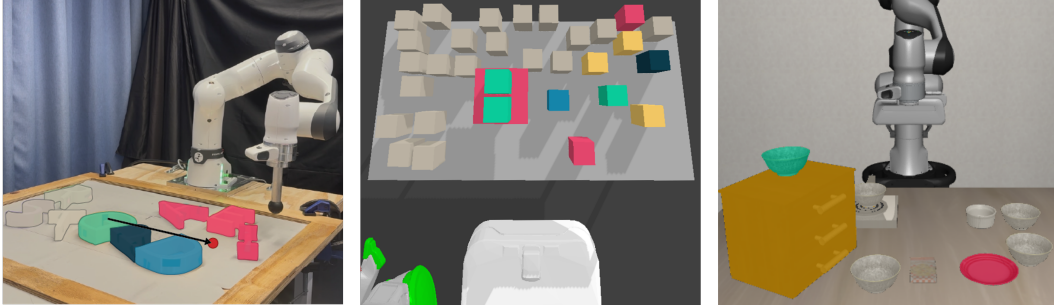


Figure 1: Scene complexity is reduced via pruning and merging. (Left) The control task is to move the green “G” (green) to a target location. LENS identifies that the “S”, “T”, and “R” (gray) can be ignored; the “A”, “E”, and “I” (pink) are relevant, but can be thought of as a single object (merged); the “P” (dark blue) and “D” (light blue) are relevant and must be modeled. (Middle) for a TAMP problem, the robot must pick and move the three green cubes. LENS identifies nearby objects as relevant, but prunes the vast majority (gray). (Right) Using a VLA, the green bowl must be moved to the red plate. Other bowls clutter the scene, but LENS identifies that they can be pruned from the observation before calling the VLA.

To address this, we introduce **LLM-guided Environment Simplification (LENS)**, a general, scalable, task-agnostic method for iteratively determining and updating task relevancy using multimodal large language models (MLLM) with task-success feedback. We consider various kinds of task relevance: semantic (“what is contextually relevant to the goal?”), geometric (“what blocks a desired motion?”), and dynamic (“what will move or be affected?”). Iterative scene refinement occurs via two elementary operations shown in Figure 1: (i) *pruning*, which removes objects that can be ignored by the planner or controller, and (ii) *merging*, which groups objects that are functionally or dynamically coupled into a single composite entity. The resulting reduced scene representation preserves physical grounding while significantly reducing the combinatorial complexity faced by downstream planners and controllers.

However, zero-shot VLM predictions can suffer from hallucinations or reasoning errors, and the environment state may evolve over time. We therefore implement closed-loop re-prompting triggered by failure: feedback from an infeasible plan or control failure is passed back to the VLM to iteratively correct the abstraction.

We evaluate LENS as a plug-and-play component in multiple robotic stacks spanning task and motion planning, model based control, and learned controllers in contact-rich manipulation, demonstrating improved scalability, success rates, and computation time in cluttered environments. The core innovation lies in leveraging modern VLMs to shift the burden of scene abstraction from hand-crafted heuristics to an automated, task-agnostic semantic reasoning loop — bridging the gap between algorithms that function in structured laboratory settings and the unbounded complexity of the real world. Our contributions are:

- A general, task-agnostic, closed-loop method for estimating task relevance using vision-language models and constructing task-focused scene abstractions.
- An integration of this abstraction layer with three distinct state-of-the-art paradigms: (i) Task and Motion Planning (TAMP) [1], (ii) optimization-based contact-implicit control (C3+) [2, 3], and (iii) the $\pi_0.5$ large-scale Vision-Language-Action (VLA) model [4].
- An experimental evaluation in cluttered tabletop settings demonstrating improved scalability, robustness, and efficiency with increasing object count.

2 Related Work

VLMs have become a versatile tool for enabling robots to reason about their environment, broadly falling into three patterns: (i) generating or sequencing actions and plans, (ii) guiding/constraining

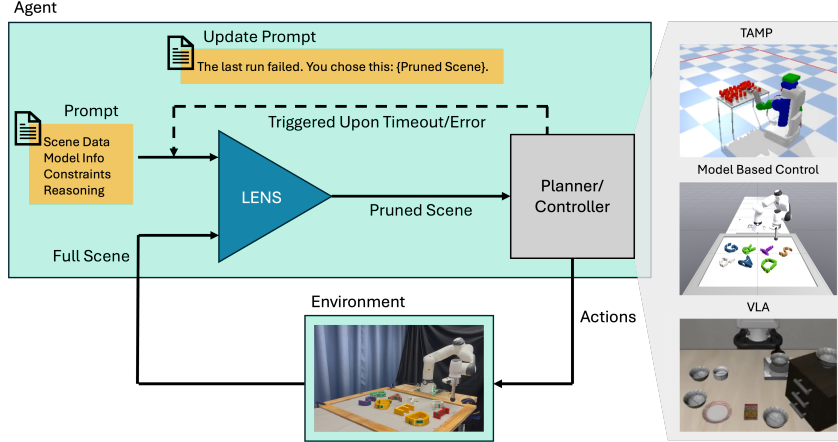


Figure 2: System overview of LENS. Given a task description via the full scene and prompt information, LENS (blue) prunes and merges objects to produce an abstracted scene, which is passed to the planner or controller (grey). When triggered, LENS revises its scene pruning. This approach generalizes to TAMP, model-based controllers, and VLAs.

planning and search, or (iii) shaping scene abstractions. Our work falls into (iii) and differs from prior approaches by actively reducing the world model before planning or control is performed.

VLMs for Action and Plan Generation. A large body of recent work uses VLMs as high-level planners, program generators, or skill sequencers [5–8]. Some methods leverage VLMs to decompose tasks into subgoals and generate executable programs [9, 10], while others directly output symbolic plans and action parameters [11]. As these methods operate on a given world representation, they are complementary to our approach of producing that task-relevant representation.

VLMs for Guiding and Constraining Planning. A related line uses VLMs or LLMs to guide classical planners by generating subgoals [12], predicting constraints [13, 14], scoring plans [15], or inferring feasibility before planning begins [16]. In all cases, the underlying planner remains unchanged and the world model is fixed. In contrast, LENS adapts the scene itself before any planning occurs, operating as a task-agnostic front-end to any downstream system.

VLM-Generated Scene Representations and Abstractions. Most relevant to our approach, several works use vision-language models or learned perception systems to build richer semantic representations of scenes, such as object-centric scene graphs, relational graphs, or affordance-based abstractions [17–22]. However, these approaches typically maintain a complete scene representation, leaving the planner or controller to contend with full combinatorial complexity.

A complementary line of work in model-based control addresses complexity through contact-implicit model reduction [23], shape approximation for improved contact tractability [24], or dynamic-resolution and lumped-object models for piles [25, 22]. These approaches, while promising, are ultimately tied to specific tasks or methods.

In visuomotor policies, language-conditioned perception [26] acts in clutter without reducing the underlying world model. Observation filtering methods [27, 28] do reduce visual inputs but require manually specified labels or fine-tuned models and do not extend to TAMP or model-based control.

3 The LENS Framework

Our approach constructs task-focused scene abstractions using a VLM, and iteratively refines these abstractions during task execution by monitoring task success. We first describe the core LENS framework, before discussing its instantiation in downstream systems in the subsequent sections.

The shared core features of LENS are shown in Figure 2 and Algorithm 1. Given a natural language task description τ and a scene description $\mathcal{O}$ as input, the VLM (in our implementation, GPT-4o) is

prompted to output information to construct a structured scene abstraction $\tilde{\mathcal{O}}$. $\mathcal{O}$ and $\tilde{\mathcal{O}}$ take a form specific to the particular planning or control framework. For example, in TAMP, $\mathcal{O}$ might consist of object-level features and predicates (e.g., mass, pose, and identity). For a VLA, it is instead an object segmentation mask overlaid on an image observation.

While the full VLM prompt is in Appendix E, we highlight the salient points here. The VLM must provide a list of lists of task-relevant objects critical to task completion to the downstream systems. Sublists are groups of objects that may be merged and treated as a single entity without hurting task performance due to functional or spatial coupling.

Given these VLM outputs, we generate the scene abstraction $\tilde{\mathcal{O}}$ by implementing “pruning” and “merging” operations on the original scene description $\mathcal{O}$ (Fig. 1), as follows.

- **Pruning.** Objects excluded from the list are omitted (“pruned”). The definition of pruning differs for each system and can range from a state space reduction in TAMP to a geometric/collision pairs reduction in C3+. At a high level, this eliminates unnecessary decision variables for TAMP, contact constraints for MPC, and visual distractors for VLA models.
- **Merging.** For each sublist of objects, LENS merges the rigid bodies into a single abstract body whose collision model is built by aggregating the members’ geometry into one co-moving description that is recomputed from the members’ poses at run time. This reduces the number of bodies and contact pairs while retaining a conservative envelope of the group’s physical extent, depending on the geometry representation used by the downstream system.

Algorithm 1 The LENS Framework

Require: Task τ , objects $\mathcal{O}$, system $\mathcal{S}$, max retries N
Ensure: Task execution via abstracted scene $\tilde{\mathcal{O}}$

```

1: for  $i = 1$  to  $N$  do
2:   Generate  $\tilde{\mathcal{O}}$  via VLM
3:   Prompt VLM from  $\tau$ ,  $\mathcal{O}$ ,  $\mathcal{S}$ ; parse output
4:   Prune & merge objects into  $\tilde{\mathcal{O}}$ 
5:   Run controller on  $\tilde{\mathcal{O}}$ 
6:   if execution succeeds then
7:     return success
8:   else
9:     Append failure feedback to prompt
10:  end if
11: end for
12: return failure

```

An important benefit of *autonomously* constructing the reduced scene representation $\tilde{\mathcal{O}}$ is that it can also be autonomously revised when necessary. In LENS, feedback is triggered by time-outs or error codes (eg. workspace limits, runtime errors) in downstream systems. Then, a framework-specific feedback prompt along with the previous $\tilde{\mathcal{O}}$ is appended to the existing prompt and current scene, and LENS re-queries the VLM for a revised scene reduction (See Fig. 2). In this manner, LENS injects high-level task semantics into a dynamic scene-abstraction process without needing manually specified task-dependent heuristics.

4 Implementing LENS In Various Control Stacks

To illustrate its versatility, we now discuss how LENS-generated scene representations $\tilde{\mathcal{O}}$ integrate into various representative downstream planning/control frameworks (more details in Appendix A).

4.1 Task and Motion Planning (TAMP) Stack

Task and Motion Planning formulates manipulation as a hybrid discrete–continuous problem, jointly solving for the sequence of parameterized actions and the parameters of those actions, which are governed by geometric and physical constraints [29]. Many TAMP frameworks operate on a domain description including a state space defined through lifted predicates [30], parameterized actions with preconditions and effects, and hybrid constraint solvers (motion planners, grasp generators, collision checkers).

A strength of TAMP lies in its explicit structure: symbolic reasoning enables planning with correctness guarantees while accounting for geometric and physical constraints. However, TAMP algo-

gorithms typically search through the decision tree generated by this structure and are therefore limited to problems of modest scope. While domain-specific relevance heuristics can prune the scene in an attempt to limit complexity, they are brittle and do not generalize across task goals/layouts or scene semantics, causing rapid performance degradation in unstructured, cluttered environments. Without a principled, semantic mechanism for reducing the scene to only what is task-relevant before planning begins, TAMP cannot be viably deployed outside of curated, low-clutter settings. An anecdotal example of VLM-based pruning vs a geometric heuristic is shown in Appendix B.

LENS Implementation. $\mathcal{O}$ is a graph of objects with poses, masses, and relations. $\tilde{\mathcal{O}}$ defines which objects are treated as active decision variables in PDDLStream [1]. Pruned objects participate in collision checking but have fixed poses and cannot be acted upon. Merged groups that must be stably movable together (e.g., a stack of objects that can be moved via the base) adopt a unified ID with the lowest supporting object as parent, and cannot be separated during planning. Feedback triggered at 120s prompts expansion of the object selection.

4.2 Contact-Implicit Model Predictive Control Stack

Contact-implicit trajectory optimization and MPC formulate manipulation as a continuous optimization problem that jointly reasons over robot motion, object dynamics, and contact interactions [2]. This enables controllers to reason about making/breaking contact, pushing multiple objects, and exploiting incidental interactions. However, each object introduces new decision variables, collision constraints, complementarity conditions, and contact pairs. The resulting problems are nonconvex and scale poorly with object count.

In cluttered scenes, this creates a fundamental barrier to real-world deployment. Controllers that simplify contact modeling, whether by limiting contact pairs [23] or softening constraints [31], reduce computational burden at the cost of physical fidelity, producing plans that fail on contact-rich tasks. Alternatively, modeling all interactions becomes intractable as object count grows. Neither path scales: the first sacrifices correctness, the second sacrifices tractability. A principled front-end that reduces the scene to its task-relevant structure is a prerequisite for deployment outside the lab.

LENS Implementation. The scene $\mathcal{O}$ is a set of objects with poses and names. Unlike TAMP, merged groups need not be physically stable; objects are clustered if their contact interactions can be reasoned about collectively (e.g., a pile to be pushed aside). The merged body’s geometry is updated in real time as the scene evolves, though the controller does not model internal shape changes. Pruning reduces the number of bodies, contact pairs, and linear complementarity problem dimensionality. Feedback is triggered at 250 control iterations or on an error code.

4.3 Vision-Language-Action Model Stack

Vision-Language-Action models combine visual encoders and language understanding modules to predict actions from perceptual observations and task descriptions [4]. These models excel at capturing semantic relationships, object affordances, and task intent, and can generalize to novel objects and instructions without explicit symbolic modeling.

However, VLAs typically operate over dense, unstructured perceptual representations. Scene understanding is implicit and distributed across learned feature spaces. So, VLAs lack explicit mechanisms for enforcing physical constraints, reasoning long-horizon, or guaranteeing task feasibility. In cluttered environments, these limitations compound: irrelevant objects, occlusions, and novel contact configurations introduce distribution shifts that corrupt affordance predictions and produce physically implausible actions.

LENS Implementation. $\mathcal{O}$ is an overhead image with object bounding boxes. Merging is not used; $\tilde{\mathcal{O}}$ is a flat list of relevant objects. Excluded objects are inpainted (See Fig. 8), producing a filtered image passed to π -0.5 [4]. This reduces visual clutter and suppresses distractor affordances without modifying the policy. Given the short task horizon, no feedback re-queries are needed.

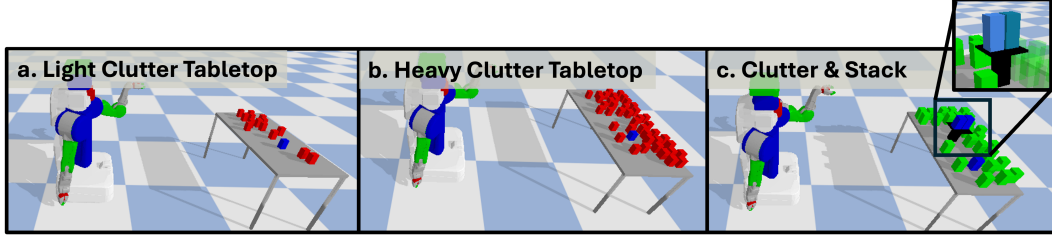


Figure 3: LENS was evaluated on three TAMP environments: (a) a lightly cluttered tabletop, (b) a heavily cluttered tabletop, and (c) a cluttered tabletop with a stack. The stack consists of a tray (black) with two goal objects (blue). In each environment, a goal object is placed on the tabletop.

5 Experiments and Results

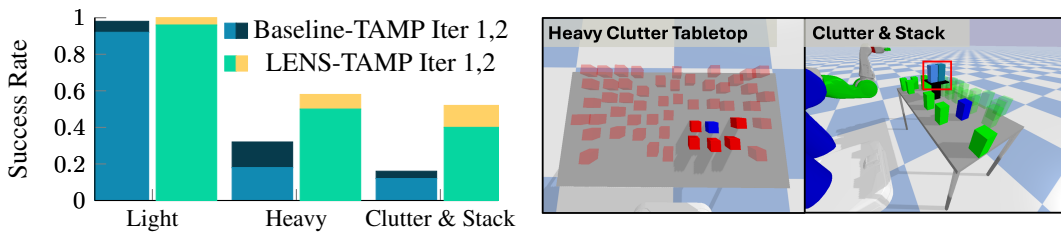
We evaluate LENS across tabletop manipulation tasks against baselines on unfiltered observations. VLM query time averaged 1.76s (model-based control, comparable times for TAMP and VLA), which is negligible relative to execution time and excluded from runtime comparisons. LENS improves success rates for all three downstream modules. In TAMP, scene pruning and object merging reduce distractor interference and enable reliable plan search under heavy clutter (Fig 4a). In model-based control, LLM-guided filtering maintains stable execution times as scene complexity scales, while the baseline degrades by orders of magnitude (Fig 5). For VLA policies, task-focused abstraction recovers substantial performance lost to visual clutter in both simulation and hardware (Table 1). These results demonstrate that a single abstraction framework generalizes across planners, and model-based and learned controllers. We analyze each set of results in the following sections.

5.1 Task and Motion Planning

To evaluate LENS’s effect of a TAMP framework’s performance, we construct three environments with increasing levels of visual and semantic complexity (Figure 3): (1) light-clutter tabletop, (2) heavy-clutter tabletop, and (3) clutter & stack scenarios. The blue object(s) need to be moved to the green goal region; the red objects can be moved if necessary. Clutter & Stack includes three target objects; two are placed on a tray, and one is placed directly on a table surrounded by clutter.

Across all environments, we compare LENS against a baseline operating on unfiltered visual observations. Performance is measured using task success rate over 50 evaluation episodes, as shown in Fig 4a. Two iterations of feedback are allowed. Success rates for each iteration are shown stacked.

In light clutter, both methods achieve comparable success; abstraction provides limited benefit when the full scene remains tractable. More noticeably in heavy clutter, LENS-TAMP outperforms the baseline, which frequently times out enumerating distractors. LLM/VLM-guided filtering suppresses irrelevant objects, improving success rates (Fig 4b). In Clutter & Stack, LENS again outperforms the baseline, which fails due to distractor interference. Merging the stack into a single composite simplifies the scene and enables more reliable plan search.



(a) In each bar in the top row, Iteration 1 and 2 success rates are stacked. Baseline is rerun twice for fairness. (b) Goal objects are blue. Semi-transparent objects are pruned. The stack is merged into one entity.

Figure 4: TAMP Results. (a) shows Baseline-TAMP and LENS-TAMP success rates in three environments arranged left to right in order of complexity: Light Clutter, Heavy Clutter, and Clutter&Stack. (b) Examples of scene pruning are shown for Heavy Clutter and Clutter&Stack.

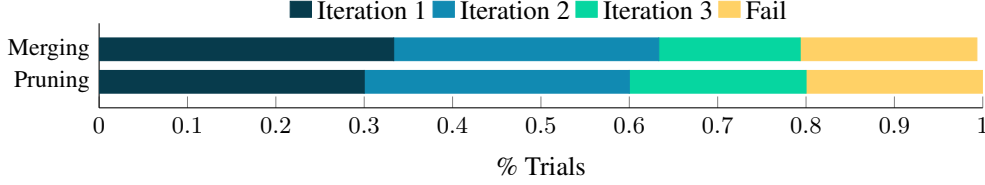


Figure 6: LENS’s task outcomes are shown for Merging and Pruning experiments on hardware. Task successes are recorded for across iterations of VLM queries.

5.2 Model-Based Control

We study a planar pushing task (Fig 7) inspired by Push Anything [3]. The goal is for the robot’s end-effector to push a goal object (“G”) to a target pose (transparent “G”). This requires reasoning over object-object, object-robot, and object-ground contact interactions. The scene is cluttered with additional objects compared to Push Anything [3], which was limited to four objects at a time.

Simulation Experiments: We varied the number of objects present in the environment and compared against the baseline controller C3+ that operates over the full scene. Baseline-C3+ does not perform any relevance estimation and reasons over all objects uniformly. Figure 5 reports average execution time vs clutter count. Results for each object are averaged across five trials. LENS-C3+ and baseline succeeded on 39/45 and 17/30 trials respectively.

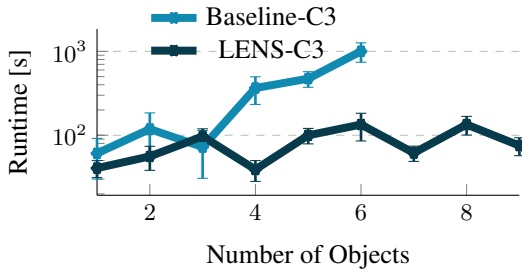


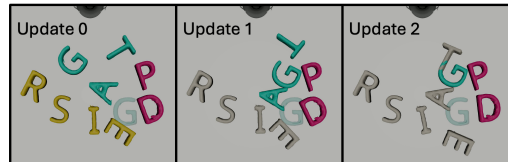
Figure 5: Object Count Ablation. As the object count increases, the baseline execution time increases drastically. LENS remains constant.

For small scenes (2–4 objects), the baseline and LENS exhibit comparable performance. In this regime, the abstraction overhead provides limited benefit as the full scene remains tractable for the underlying planner and controller. As object count increases, however, baseline-C3+ sharply degrades in performance. At 6 objects, baseline-C3+ becomes prohibitively slow, exceeding LENS by an order of magnitude (~ 1000 seconds). At 7 objects, it reaches over 4000 seconds. In contrast, LENS-C3+ maintains stable performance across all tested clutter levels, with average execution time of ~ 40 –135 seconds. This indicates that the effective problem size is determined by task relevance rather than raw scene complexity. We also found LENS had higher success rates compared to various distance-based geometric pruning baselines (Appendix C).

Real Robot Experiments: We evaluate two hardware variations: pruning only (precise pushing goal object “G” to a target pose and orientation) and pruning+merging (gross pushing through clutter with a looser position-only threshold). Fig 6 shows the success rate broken down by iteration, limited to three feedback loops. For both pruning and merging, the cumulative success rate is high at 80%, with successes distributed throughout all three iterations rather than concentrated in the first. This suggests that aggressive initial pruning may omit relevant contacts, with subsequent VLM feedback correcting for this and enabling task completion. Failure cases include timeouts



(a) Pruned objects are shown in gray. Successive iterations refine the abstraction to adapt to failures, mistakes, or evolving environment states.



(b) Merged groups, shown with uniform colors, become more focused over iterations.

Figure 7: Three iterations of (a) pruned and (b) merged scene graph at various times during an episode of multi-object planar pushing (Sec 5.2). Goal location for target “G” is overlaid.



Figure 8: Real hardware VLA experiment. LENS is applied directly on hand image (b) without simulator bounding boxes. Foundation models [32–34] are used to segment and inpaint scenes (c). Performance improvements persist, supporting the validity of the simulation comparison.

and workspace limits. A visualization of three feedback loops on hardware is shown in Fig 7. These results demonstrate that iterative VLM feedback enables recovery from execution errors while maintaining high success on hardware in model-based control.

5.3 Vision Language Action Models

Simulation Experiments: We next evaluate task-focused scene abstraction with a Vision–Language Action (VLA) using the Spatial LIBERO benchmark with a $\pi_{0.5}$ policy. Spatial LIBERO consists of tabletop manipulation tasks that require reasoning over relative spatial relationships, such as placing, stacking, or aligning a single object type (bowl) in varying positions across different scenes. To stress test semantic and spatial reasoning, we introduce additional instances of the goal object beyond the original Spatial LIBERO scenes, increasing visual ambiguity and adding distractions. An example scene and VLM pruning selection is included in Appendix D. Ground-truth segmentation is used in simulation as open-vocabulary detectors perform unreliably under domain mismatch [32].

Task success rates averaged over 100 evaluation episodes across 10 tasks (Appendix D) show that clutter significantly degrades VLA performance, with success dropping from 0.85 to 0.5, in keeping with prior reports [35–37]. LENS recovers performance substantially, improving success to 0.69. The remaining gap highlights limitations of imperfect abstraction and the underlying VLA’s robustness to distribution shift. Overall, these results indicate that task-focused scene abstraction is an effective, lightweight mechanism for improving VLA performance in visually cluttered spaces.

Real Robot Experiments: On hardware, experiments (Figure 8, Table 1) do not have privileged bounding boxes. An object detection and inpainting pipeline (Appendices D, E) is used. $\pi_{0.5}$ picks up one of six fruit plushies and place it in a target bowl. Performance gains persist, confirming they reflect scene abstraction.

Method	Success Rates			
	Pineapple	Pear	Banana	Apple
LENS in Clutter	0.5	0.3	0.3	0.7
Baseline in Clutter	0.2	0.0	0.0	0.0

Table 1: Hardware VLA ($\pi_{0.5}$) results (n=10 per fruit). LENS consistently improves over the baseline across tasks.

6 Conclusion

We introduced LENS, a general, task-agnostic framework that constructs compact, physically grounded scene abstractions by pruning irrelevant objects and merging functionally coupled ones using VLMs and task feedback. Integrated as a front-end to existing pipelines, LENS improves scalability, robustness, and efficiency across contact-rich, cluttered manipulation scenarios.

Limitations. While abstract representations could be validated in simulation before execution, it is computationally prohibitive for some controllers. The feedback mechanism relies on high-level failure signals rather than explicit causes, and LENS does not learn from experience across runs.

More broadly, this work argues that as robots are deployed in increasingly complex and unstructured environments, the ability to dynamically construct minimal, task-conditioned world models will be critical for scaling both model-based and learning-based methods. Future work can extend these ideas to richer abstraction operations (eg. via RL, finetuning), tighter perceptual integration, and long-horizon multi-stage tasks where the notion of relevance evolves over time.

Acknowledgments

This work was supported by DARPA TIAMAT under Grant No.HR0011249042, NSF CAREER Awards under Grant Nos. 2238480 and 2239301, the Office of Naval Research (ONR) under Grant No. N00014-22-1-2677, and NSF Science of Learning and Embedded Systems (SLES) program under Grant No. 2331783.

References

- [1] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *Proceedings of the international conference on automated planning and scheduling*, volume 30, pages 440–448, 2020.
- [2] A. Aydinoglu, A. Wei, W.-C. Huang, and M. Posa. Consensus complementarity control for multi-contact mpc. *IEEE Transactions on Robotics (TRO)*, July 2024. doi:10.1109/TRO.2024.3435423. URL <https://ieeexplore.ieee.org/document/10614849>.
- [3] H. Bui*, Y. Gao*, H. Yang*, E. Cui, S. Mody, B. Acosta, T. S. Felix, B. Bianchini, and M. Posa. Push anything: Single- and multi-object pushing from first sight with contact-implicit mpc. *arXiv preprint arXiv:2510.19974*, 2025.
- [4] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [5] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- [6] Y. Yang, J. Sun, S. Kou, Y. Wang, and Z. Deng. Lohovla: A unified vision-language-action model for long-horizon embodied tasks. *arXiv preprint arXiv:2506.00411*, 2025.
- [7] Y. Feng, J. Han, Z. Yang, X. Yue, S. Levine, and J. Luo. Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation. In J. Lim, S. Song, and H.-W. Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 2038–2062. PMLR, 27–30 Sep 2025. URL <https://proceedings.mlr.press/v305/feng25b.html>.
- [8] F. Lin, R. Nai, Y. Hu, J. You, J. Zhao, and Y. Gao. Onetwovla: A unified vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2505.11917*, 2025.
- [9] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can and not as i say: Grounding language in robotic affordances. In *arXiv preprint arXiv:2204.01691*, 2022.
- [10] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *arXiv preprint arXiv:2209.07753*, 2022.
- [11] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu. Llm³: Large language model-based task and motion planning with motion failure reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12086–12092. IEEE, 2024.
- [12] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling. Guiding long-horizon task and motion planning with vision language models, 2024. URL <https://arxiv.org/abs/2410.02193>.
- [13] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett. Open-world task and motion planning via vision-language model inferred constraints. *arXiv preprint arXiv:2411.08253*, 2024.

- [14] D. Lee, S. Joo, K. Lee, and B. Kim. Prime the search: Using large language models for guiding geometric task and motion planning by warm-starting tree search. *The International Journal of Robotics Research*, page 02783649251347307, 2024.
- [15] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- [16] M. Yan, M. Mengdibayev, A. Floros, W. Guo, L. E. Kavraki, and Z. Kingston. Using vlm reasoning to constrain task and motion planning. *arXiv preprint arXiv:2510.25548*, 2025.
- [17] S. Nguyen, O. Oguz, V. Hartmann, and M. Toussaint. Self-supervised learning of scene-graph representations for robotic sequential manipulation planning. In J. Kober, F. Ramos, and C. Tomlin, editors, *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 2104–2119. PMLR, 16–18 Nov 2021. URL <https://proceedings.mlr.press/v155/nguyen21b.html>.
- [18] Y. Zhu, J. Tremblay, S. Birchfield, and Y. Zhu. Hierarchical planning for long-horizon manipulation with geometric and symbolic scene graphs. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6541–6548. Ieee, 2021.
- [19] T. Silver, R. Chitnis, A. Curtis, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling. Planning with learned object importance in large problem instances using graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, number 13, pages 11962–11971, 2021.
- [20] Y. Zhang, T. Xue, A. Razmjoo, and S. Calinon. Learn2decompose: Learning problem decomposition for efficient sequential multi-object manipulation planning. *arXiv preprint arXiv:2408.06843*, 2024.
- [21] C. V. Braun, J. Ortiz-Haro, M. Toussaint, and O. S. Oguz. Rhh-lgp: Receding horizon and heuristics-based logic-geometric programming for task and motion planning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13761–13768. IEEE, 2022.
- [22] J. Qian, Y. Li, B. Bucher, and D. Jayaraman. Task-oriented hierarchical object decomposition for visuomotor control. *arXiv preprint arXiv:2411.01284*, 2024.
- [23] H. Bui and M. Posa. Enhancing task performance of learned simplified models via reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9212–9219. IEEE, 2024.
- [24] N. Nechyporenko, Y. Zhang, S. Campbell, and A. Roncone. Morphit: Flexible spherical approximation of robot morphology for representation-driven adaptation. *arXiv preprint arXiv:2507.14061*, 2025.
- [25] Y. Wang, Y. Li, K. Driggs-Campbell, L. Fei-Fei, and J. Wu. Dynamic-resolution model learning for object pile manipulation. *arXiv preprint arXiv:2306.16700*, 2023.
- [26] M. Shridhar, L. Manuelli, and D. Fox. Cliport: What and where pathways for robotic manipulation. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, 2021.
- [27] R. Mirjalili, T. Jülg, F. Walter, and W. Burgard. Augmented reality for robots (arro): Pointing visuomotor policies towards visual robustness. *IEEE Robotics and Automation Letters*, 2026.
- [28] J. Zhang, M. Memmel, K. Kim, D. Fox, J. Thomason, F. Ramos, E. Bıyık, A. Gupta, and A. Li. Peek: Guiding and minimal image representations for zero-shot generalization of robot manipulation policies. *arXiv preprint arXiv:2509.18282*, 2025.

- [29] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- [30] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins. *PDDL-the planning domain definition language*, 1998.
- [31] S. Le Cleac’h, T. A. Howell, S. Yang, C.-Y. Lee, J. Zhang, A. Bishop, M. Schwager, and Z. Manchester. Fast contact-implicit model predictive control. *IEEE Transactions on Robotics*, 40:1617–1629, 2024.
- [32] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European conference on computer vision*, pages 38–55. Springer, 2024.
- [33] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- [34] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park, and V. Lempitsky. Resolution-robust large mask inpainting with fourier convolutions. *arXiv preprint arXiv:2109.07161*, 2021.
- [35] A. Rasouli, M. Alban, S. Pakdamansavoji, Z. Li, Z. Zhang, A. Wu, and X. Zhao. Distracted robot: How visual clutter undermine robotic manipulation. *arXiv preprint arXiv:2511.22780*, 2025.
- [36] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [37] B. Zhang, J. Li, J. Shen, Y. Cai, Y. Zhang, Y. Chen, J. Dai, J. Ji, and Y. Yang. Vla-arena: An open-source framework for benchmarking vision-language-action models. *arXiv preprint arXiv:2512.22539*, 2025.
- [38] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

A Background Details

The abstraction formatting and rules vary across integrations to match the physical and representational requirements of each downstream system. The core LENS pipeline is unchanged; these differences reflect how $\tilde{\mathcal{O}}$ is expressed in a form each module can consume.

A.1 LENS For Task and Motion Planning (TAMP)

We represent the environment $\mathcal{O}$ as a scene graph of objects with known poses, masses, relations (such as stacked, on) and properties (such as graspable). $\tilde{\mathcal{O}}$ is the subset of $\mathcal{O}$ whose poses are treated as decision variables in the planning problem if PDDLStream [1], allowing actions that explicitly modify their pose (e.g., pick, place, or push). In contrast, static objects excluded from $\tilde{\mathcal{O}}$ have fixed poses and cannot be acted upon, though they still impact feasibility.

To determine $\tilde{\mathcal{O}}$, LENS is provided with a list of object IDs and their associated mass and pose, along with a goal specification represented as a list of tuples (o_i, g_i) , where each tuple corresponds to an object and its desired goal pose.

Sublist groups must be stably movable together (eg. stacks). They adopt a new, unified ID, and the parent object is the lowest supporting object in the stack. Sublist groups have their geometries fused and maintain the mass of the base object. A set of merged objects cannot be moved separately. To maintain feasibility constraints, collision checking over the full scene $\tilde{\mathcal{O}}$ is still required.

In LENS, feedback is triggered when no solution has been found or when the maximum timeout of 120 seconds has been reached. The feedback includes the previous scene representation and a description of the failure in the form of: "The last run failed. You chose [previous representation]. Choose a larger set of objects from the scene that is not just goal objects." This feedback encourages larger selection sets to expand the plan search. The feedback is appended to the original prompt and new scene representation.

A.2 LENS for Model-Based Control

We represent the scene as a set of objects $\mathcal{O}$. Given the set of object's poses and names, LENS outputs $\tilde{\mathcal{O}}$, the reduced set of objects.

Note that sublist sets are not required to be stably movable and are instead clustered if their contact sampling can be reasoned about together. For example, piles of objects are not physically stable but can be pushed aside together. The collective union shape of the sublist sets are updated in realtime as the scene changes. Since the controller treats the pile as a single object, it does not model or predict how its shape will shift over time.

We integrate this task-focused scene abstraction $\tilde{\mathcal{O}}$ into the C3+ contact-implicit controller [3] that models manipulation as a linear complementarity system (LCS) and solves a linear complementarity problem (LCP) at each control step. Selected objects contribute a rigid body to the plant, potential contact pairs, and sampled surface regions. Pruning reduces the number of bodies, contact pairs, and LCP dimensionality, including the surface sampling space. Both contact generation and contact resolution therefore operate on a reduced-order system.

Feedback is triggered when 250 controller loop iterations was reached or an error code was received. The feedback includes the previous scene representation and a description of the failure in the form of: "Your previous answer was not solvable. You selected [past object selection]." The feedback is appended to the original prompt and new scene representation.

A.3 LENS For Vision Language Action Model

We integrate task-focused scene abstraction into a vision-language-action (VLA) policy by filtering visual inputs using the VLM output. Before querying the VLA (π -0.5 trained on the LIBERO-Spatial dataset [38]), the VLM is provided with $\mathcal{O}$, an overhead scene image and ground-truth object

bounding boxes from the simulator, a semantic goal sentence, and the original prompt. $\tilde{\mathcal{O}}$ does not include sublists for the VLA.

To construct an input for the VLA using $\tilde{\mathcal{O}}$, a new image is created with only the objects in $\tilde{\mathcal{O}}$ included. This is implemented via generative-modeling based image inpainting, yielding less cluttered, task-focused observations. Details are found in Appendix D. The resulting filtered images are then used as input to the VLA policy. This preprocessing step reduces visual clutter and semantic ambiguity by suppressing distractor objects unrelated to task execution. As a result, the VLA receives observations that are more closely aligned with the task-relevant scene structure encountered during training, improving robustness in cluttered environments. Beyond the training distribution, this filtering reduces the number of visually plausible affordances in the scene, simplifying action selection under clutter. Given the short task horizon, the filtered scene remains valid throughout execution, so no additional VLM re-queries are needed.

B TAMP Simulation Examples

An anecdotal example (Figure 9) showcases an iteration of LENS pruning compared to a distance based metric such as only planning over the closest 4 target objects. While LENS may not be perfect and may have spurious selections, it is able to maintain the critical objects when multiple blocking objects need to be removed for access.

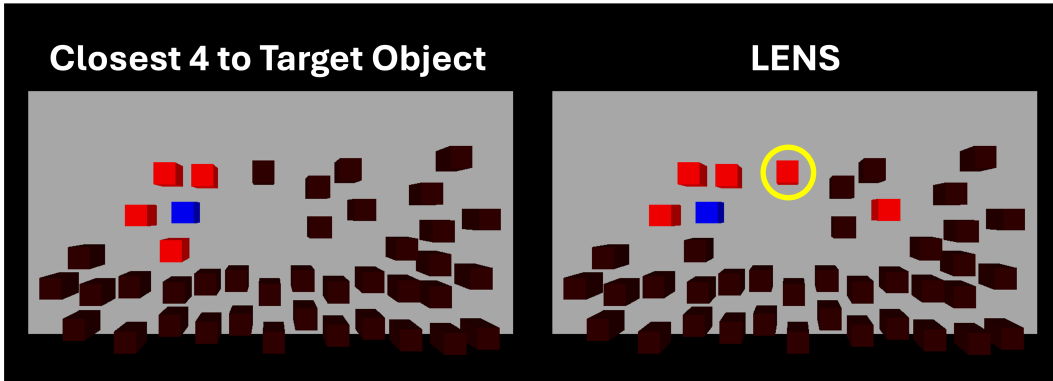


Figure 9: Example TAMP scene where a distance-based baseline fails. The set of nearest objects to the goal (highlighted) fails to include a critical blocking object further away (circled yellow), which must be removed to enable a successful approach to the target. LENS identifies the blocker as task-relevant.

C C3+ Simulation Baseline Comparisons

We compared LENS to distance-based geometric pruning baselines for C3+ (Figure 10). These baselines select the 3 spatially nearest objects or the objects within r m radius to the goal and pass them to the downstream system. Results show that geometric proximity is a reasonable heuristic, but task success may require more distant objects for multi-step planning. This supports the claim that the VLM’s task-relevant reasoning adds value beyond simple spatial heuristics, which typically require human-engineering and fine-tuning.

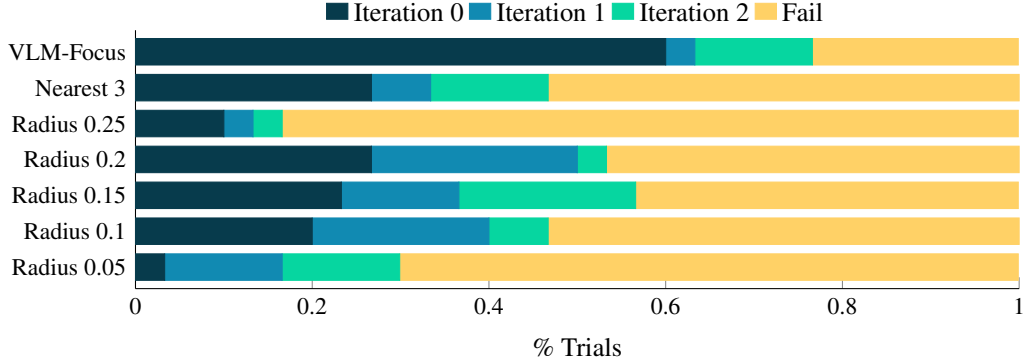


Figure 10: C3+ distance-based pruning baseline (n=30). Nearest-3 objects or nearest objects in r radius are passed to C3+. LENS outperforms the heuristics by capturing task-relevant objects regardless of proximity.

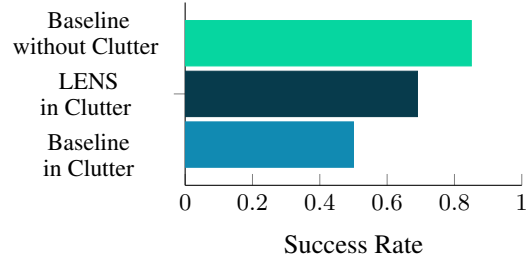
D VLA Experiments

In simulation, object masking is implemented using ground-truth segmentation rather than learned detectors. This choice is motivated by the observation that open-vocabulary detectors such as GroundingDINO [32] perform unreliably in simulated scenes due to domain mismatch, leading to degraded masking quality unrelated to the abstraction mechanism itself.

In hardware, LENS selections are passed through GroundingDINO [32], SegmentAnything [33], and LaMa [34] to inpaint over task-irrelevant object masks. A hardware-specific version of the VLM prompt (see Appendix E) was used to produce outputs compatible with the GroundingDINO detection pipeline.



(a) VLA Pruning Visualization. Grey objects in the scene have been pruned away, while objects with colored mask overlays are identified as semantically and spatially goal relevant. The goal (masked green) and target objects (masked purple) are selected. In addition, a second distractor goal is selected due to ambiguity in the prompt.



(b) The VLA is executed directly on the cluttered scene in Baseline in Clutter. In LENS, task-irrelevant objects are removed using our VLM-based scene abstraction prior to querying the VLA. Baseline without Clutter corresponds to the original, uncluttered LIBERO setup.

E Prompts

Condensed VLM Prompt

You are a precise symbolic robot planner assistant. Your task is to categorize objects into lists based on physical properties and spatial relations.

SCENE DATA

- Environment: You are operating in a cluttered tabletop scene.
- Goal: {goal}
- Objects: {objects}
- Stacked: {stacked}

MODEL INFORMATION

You are filtering the scene for a TAMP planner. This is a tabletop scene with multiple goal objects.

List [2] object groupings MUST be stably movable together (cannot just be a cluster).

RULES

1. Include ALL goal objects and any clutter obstructing or near them. Do NOT omit objects between robot and goal.
2. Each sublist groups objects that can be stably moved together (not just spatial clusters).
3. Objects sharing the same base must be combined into one sublist — base object listed first.
4. Standalone relevant objects with no group are listed as singletons: [obj].
5. Order sublists by task importance.

REASONING STEPS

1. **Goals:** Which objects are part of 'goal'?
2. **Obstructions:** Which objects are stacked on, supporting, or blocking a goal object?
3. **Grouping:** Which objects can move together? Merge any that share a base.
4. **Order:** Base first within each sublist. Most important sublist first overall.

OUTPUT FORMAT

Reasoning: one line, 30 tokens

Result: [[base, obj, ...], [obj], [base, obj, ...], ...]

FINAL OUTPUT

Reasoning:

Result:

TAMP Cluttered Tabletop

You are filtering the scene for a TAMP planner. The robot is a PR2 movable base robot located at pos = [2.5, 0, 0].

This is a cluttered tabletop scene with one goal object. You should reduce clutter to reason over. Think about what would obstruct robot from goal object.

TAMP Clutter & Stack

Filtering a cluttered tabletop scene for a TAMP planner. Robot is a PR2 at pos = [2.5, 0, 0]. There is clutter around EACH goal object and on the table. DO NOT FORGET THERE IS BOTH TRAY GOAL OBJECTS AND TABLE GOAL OBJECTS. You need to list clutter around all beyond just goal objects! Clutter can be singletons.

C3+ Pruning

You are a model based controller. You are trying to reduce number of objects to reason over to reduce linear complementarity problem size and contact constraints. Mergings in this model are object groupings (excluding the goal object) to be reasoned about together for computational efficiency. They are encouraged. **CHOOSE AT MOST 4 objects!** Narrow your selection to less objects when closer to the goal

C3+ Merging

You are a model-based controller optimizing a Linear Complementarity Problem (LCP). Computational cost scales directly with the number of sublists. Mergings in this model are object groupings to be reasoned about together for computational efficiency. They are encouraged.

IMPORTANT — THIS SECTION SUPERSEDES RULES AND REASONING STEPS BELOW.

DO NOT LIMIT MERGING to physically stacked or physically grouped objects, any clutter can be merged. Ignore base sharing requirements.

Merging means placing multiple object names inside the same sublist: [obj a, obj b, obj c]. This is one sublist, not three. A result with 9 singletons has 9 sublists — that is the **WORST** possible outcome.

BAD: [[G], [A], [B], [C], [D], [E], [F], [H], [I]] — 9 sublists, maximum LCP cost

GOOD: [[G], [A, B, C, D], [E, F, H, I]] — 3 sublists, minimum LCP cost

Completely irrelevant non-goal objects can be entirely ignored too and not listed as merges or singletons. Keep goal object as separate sublist.

MERGES ARE STRONGLY ENCOURAGED FOR NON GOAL OBJECTS! When goal object is close to target, singletons are preferred.

AT MOST 4 SUBLISTS TOTAL!!!!!! DO NOT GIVE MORE THAN 4 SUBLISTS!!!!

Libero Spatial

You are filtering a cluttered scene for a pi0.5 VLA. You are trying to **SEMANTICALLY/SPATIALLY SIMPLIFY THE SCENE**. Focus on what goal semantics say and spatial relevancy of objects. Remove objects that would confuse the goal! Keep in distribution with trained Libero-10 dataset. Keep target location and specific goal object! There is one target object.

You are helping a robot manipulation pipeline clean up camera images.

The robot's current task is: "{goal}"

Look at the robot workspace image(s) and identify:

1. Objects **RELEVANT** to the task (to be **PROTECTED** from removal)
2. **DISTRACTOR** objects on the workspace (to be **REMOVED** from the image)

Rules for **RELEVANT** objects (protect):

- ALWAYS include: the robot arm, gripper, and end effector (if visible). The gripper/end effector is the most critical — it includes the finger pads, gripper jaws, and anything the robot is currently grasping. NEVER remove it.
- ALWAYS include: any object the robot needs to interact with for the task
- ALWAYS include: any container/bowl/plate the target object is sitting in or on
- ALWAYS include: any goal location (basket, bin, tray, bowl) mentioned in the task
- DO NOT include: background furniture (table, shelf, wall, floor)

Rules for **DISTRACTOR** objects (remove):

- List **EVERY** single visible movable object that is **NOT** the task target or robot
- Include each fruit, toy, container, bowl, plate separately — do NOT skip any
- If an object is inside a bowl/plate, list **BOTH** the object **AND** the bowl/plate
- Even if you are unsure whether something is relevant, include it as a distractor unless the task description specifically mentions it
- DO NOT include: the robot arm, gripper, end effector, or task-relevant objects
- DO NOT include: background furniture, table, shelf, wall, floor

Respond with **ONLY** a JSON object, no explanation:

```
{
  "relevant_objects": [
    {
      "label": "short name",
      "visual_description": "color, shape, material
      (e.g. orange-red round plush)"
    }
  ],
  "distractors": [
    {
      "label": "specific name",
      "visual_description": "color, shape, material"
    }
  ]
}
```

CRITICAL labeling rules:

- Use the **SAME** object names as the task description when possible. If the task says "apple", label it "apple" — NOT "orange fruit" or "red ball".
- These may be toy/fake versions of objects. A toy apple is still "apple".
- For distractors, be **SPECIFIC**: use color + object type (e.g. "yellow banana toy", "purple plum toy", "teal bowl") so each distractor can be individually detected.
- visual_description should describe what the object **LOOKS LIKE** in the image (color, shape, texture) — this helps the detection model find the right object.
- Only include objects that are **EXPLICITLY** part of the task as relevant.
- Do NOT include relevant objects in the distractors list.
- It is **CRITICAL** that you list **ALL** distractors. Missing even one is a failure.

Example relevant: "robot arm", "apple"

Example distractors: "pineapple toy", "yellow banana toy", "purple plum toy", "orange slice toy", "teal bowl", "gold bowl", "dark blue bowl", "pink bowl"